Task Management System — Architecture & Execution Report

System Flow, State Sharing, Action Tracing & Error Handling Assessment | Prepared by SJ Motsemme

---

Question 1: Application Startup, API Task Retrieval & Local Cache Synchronization

The application initialization and data synchronization sequence follows a clean, lifecycle-aware reactive pattern connecting the Flutter frontend with the C# .NET Web API and SQLite local database.

1. Entry Point & Dependency Injection (main.dart):

In main.dart, main() invokes runApp(const MyApp()). Inside MyApp.build(), the HTTP client service (TaskApiService) and local SQLite storage service (TaskCacheService) are instantiated and injected into TaskRepository. A MultiProvider tree registers ThemeNotifier, Provider<TaskRepository>, and ChangeNotifierProvider<TaskListViewModel> at the root.

2. Post-Frame Mount Trigger (task_list_view.dart):

When TaskListView mounts, _TaskListViewState.initState() schedules context.read<TaskListViewModel>().fetchTasks() using WidgetsBinding.instance.addPostFrameCallback(...). This guarantees that initial API calls execute immediately after the first frame renders, preventing setState() calls during build.

3. Reactive ViewModel State Transition (task_list_view_model.dart):

Inside TaskListViewModel.fetchTasks(), state transitions to ViewState.loading and notifyListeners() triggers the UI loading spinner. It then awaits _repository.getTasks().

4. Repository Fetch & SQLite Cache Sync (task_repository.dart & task_cache_service.dart):

TaskRepository.getTasks() calls _apiService.fetchTasks(), issuing an HTTP GET to http://localhost:5164/api/Tasks. Upon receiving HTTP 200, the JSON payload is parsed into List<Tasks> objects. The repository immediately invokes _cacheService.cacheTasks(remoteTasks), which executes an atomic SQLite transaction (clearing stale records via DELETE FROM tasks and inserting fresh records via INSERT INTO tasks). On Web platforms, it populates _webInMemoryCache.

5. Final Loaded Render & Offline Fallback:

Upon successful receipt of remote tasks, TaskListViewModel updates state to ViewState.loaded and calls notifyListeners(), causing the UI to re-render the task list. If the network is unavailable, TaskRepository.getTasks() catches the exception and falls back to _cacheService.getCachedTasks(), serving offline cached data seamlessly.

Question 3: Mobile & Desktop Views Shared State & Behavior Architecture

The application implements a decoupled, single-source-of-truth architecture where layout presentation is separated from state management and business logic.

Architectural Pillar	Implementation Details
Single Source of Truth	Both TaskListMobileView and TaskListDesktopView listen to the exact same TaskListViewModel instance provided at the root by MultiProvider.

Base View Decision	TaskListView uses a LayoutBuilder (breakpoint width = 800px) to determine whether to render mobile or desktop layouts dynamically without duplicating ViewModel instances.
Zero Logic Duplication	All business actions (date selection, category management, priority filtering, task creation, status updates, deletion, and Eye Comfort Shield theme toggles) execute directly through shared ViewModel methods.
Responsive Layout Adaptation	Desktop view anchors a permanent 300px side navigation rail on wide screens, embedding the shared task viewport while mobile view renders a collapsible drawer.

Question 4: Action Trace — Creating a Task from View to Backend & Cache Update

The following trace details the execution path when a user creates a new task *'Review Architecture'*:

Layer / Stage	Execution Path & Operation
1. View Layer *(TaskListMobileView)*	User enters Title, Description, Priority, Due Date, Category in the bottom sheet and taps **Save Task**. View validates non-blank title and executes: context.read<TaskListViewModel>().addTask(title, desc, priority, dueDate, category)
2. ViewModel *(TaskListViewModel)*	Constructs a Tasks object (ID = 0, current timestamp) and calls _repository.addTask(newTask).
3. Repository *(TaskRepository)*	Invokes _apiService.createTask(newTask).
4. API Service *(TaskApiService)*	Serializes task to JSON (task.toJson()) and executes POST http://localhost:5164/api/Tasks with Content-Type: application/json.
5. Backend Controller *(TasksController.cs)*	ASP.NET Core binds JSON payload via [JsonPropertyName(...)] annotations to C# TaskItem model and executes TasksController.Create(task).
6. C# Repository *(InMemoryTaskRepository.cs)*	Assigns auto-increment ID (_nextId++), sets CreatedAt = DateTime.UtcNow, stores in _tasks list, and returns HTTP 201 Created with JSON payload.
7. Cache & UI Refresh *(TaskCacheService & View)*	TaskApiService parses HTTP 201 JSON. TaskRepository triggers getTasks(), updating local SQLite database. TaskListViewModel updates state to loaded and calls notifyListeners(), re-rendering the View with the new task.

Question 5: Client Error Handling — Validation, 404, Server & Connectivity Failures

The client application implements multi-layered exception handling to guarantee system stability and transparent error messaging for both client-side and server-side failures:

1. Client-Side Input Validation (400 Bad Request):

In the View layer, task forms validate title inputs before network dispatch. If blank or whitespace-only, the UI intercepts submission and displays a **400 Bad Request Error**:

Client-Side Error (400 Bad Request): Task Title cannot be empty or blank.

2. Categorized HTTP Error Parsing (TaskApiService.cs):

HTTP response status codes are evaluated and mapped into clear, human-readable exception messages:

- **HTTP 400 (Bad Request):** Client-Side Issue: Invalid task details provided. Please check your input.
- **HTTP 404 (Not Found):** Client-Side Issue: The requested task could not be found.
- **HTTP 401/403 (Unauthorized):** Client-Side Issue: Access denied by server.
- **HTTP 500 / 5xx (Server Error):** Server-Side Issue (Code 500): The server encountered a problem. Please try again later.

3. Network & Timeout Handling:

Network exceptions (SocketException, ClientException, XMLHttpRequest) are caught and mapped to:
Client-Side Issue: Network or CORS connection error. Please verify API is running at http://localhost:5164.
Connection timeouts (TimeoutException) map to:
Client-Side Issue: Connection timed out. Server took too long to respond.

4. Centered Floating Red SnackBar Overlay (UI Layer):

When any async operation fails, ViewModels capture the exception message cleanly without crashing. The UI calls _showCenterErrorSnackBar, inserting a floating red OverlayEntry at top: 35% (center of the screen) directly in front of open bottom sheet containers. This guarantees the user is prominently informed of exact failure causes while keeping the application completely responsive.